



Queensland University of Technology
Brisbane Australia

This may be the author's version of a work that was submitted/accepted for publication in the following source:

[Ramachandran, Gowri, McDonald, Lewyn, & Jurdak, Raja](#)
(2023)

FUSE: Fault Diagnosis and Suppression with eBPF for Microservices.
In Monti, Flavia, Rinderle-Ma, Stefanie, Ruiz Cortés, Antonio, Zheng, Zibin, & Mecella, Massimo (Eds.) *Service-Oriented Computing: 21st International Conference, ICSOC 2023, Rome, Italy, November 28 – December 1, 2023, Proceedings, Part I*.
Springer, Cham, Switzerland, 243–257.

This file was downloaded from: <https://eprints.qut.edu.au/243145/>

© 2023 The Author(s)

This work is covered by copyright. Unless the document is being made available under a Creative Commons Licence, you must assume that re-use is limited to personal use and that permission from the copyright owner must be obtained for all other uses. If the document is available under a Creative Commons License (or other specified license) then refer to the Licence for details of permitted re-use. It is a condition of access that users recognise and abide by the legal requirements associated with these rights. If you believe that this work infringes copyright please provide details by email to qut.copyright@qut.edu.au

License: Creative Commons: Attribution-Noncommercial 4.0

Notice: *Please note that this document may not be the Version of Record (i.e. published version) of the work. Author manuscript versions (as Submitted for peer review or as Accepted for publication after peer review) can be identified by an absence of publisher branding and/or typeset appearance. If there is any doubt, please refer to the published source.*

https://doi.org/10.1007/978-3-031-48421-6_17

FUSE: Fault Diagnosis and Suppression with eBPF for Microservices

Gowri Sankar Ramachandran, Lewyn McDonald, and Raja Jurdak

Trusted Networks Lab
Queensland University of Technology, Brisbane, Australia

Abstract. Modern applications use microservices architecture due to its scalability, loose coupling, and abstraction benefits. Following this architecture, large applications get divided into small and composable services with API communication support and hosted on the cloud. The deployment of services on the cloud offers elastic load balancing, cost efficiency, and ease of management, but this deployment model introduces two problems; firstly, the service owner must trust third-party cloud providers to run services, all while tolerating potential faults. Secondly, the service owner cannot reliably diagnose runtime faults with generic and coarse-grained logs offered by the cloud infrastructure. Having the ability to deeply introspect microservices' runtime behaviour on a third-party cloud infrastructure would provide opportunities to improve the resiliency of cloud-native microservice-based applications significantly.

This paper introduces *FUSE*, a novel framework based on extended Berkeley Packet Filter (eBPF) technology that addresses these challenges by enabling deep introspection of microservices' runtime behaviour. FUSE observes microservices' runtime behaviour at the kernel level by tracing the system calls, function invocations, and disk accesses. Subsequently, it creates a *unique hash-based digest* from the traces for each microservice invocation. FUSE then uses the digest to verify the microservice's runtime correctness. A correct microservice repeatedly produces a known and deterministic digest, while a faulty service generates a random and unknown trace. Moreover, FUSE offers a *real-time fault detection and suppression* capability to microservices architecture and prevents cascading failures promptly. Lastly, FUSE introduces a *stability* score for microservices to capture their runtime consistencies succinctly. During evaluation using four representative microservices on AWS EC2 instances, FUSE successfully detected **84 runtime faults**.

Keywords: eBPF · Resilient Microservice · Fault Diagnosis.

1 Introduction

Contemporary applications adopt microservices architecture to achieve scalability, load balancing, continuous integration and loose coupling for large-scale enterprise applications [7]. Some of the largest tech companies, including Amazon [8] and Microsoft [12], serve millions of customers following microservices architecture. The emergence of cloud computing platforms further accelerates the growth of microservices, allowing the service owners to deploy their applications on the cloud without

needing to invest in hardware, software, and tooling resources, as existing cloud platforms offer many built-in services, including elastic load balancing, security, and remote management, for an affordable cost, making them attractive for the deployment of microservices [21].

As the applications switch from the “monolithic” to the “microservices” model, the end system consists of a set of independent services running on different cloud instances in a virtualised environment. Although cloud providers strive to provide reliable services to the service owners, there is still a possibility of service failures and faults at runtime [1, 6, 5]. Service meshes have been developed to make microservices resilient against communication failures, as the failure of a single service could impact other inter-connected services through the cascading effect [11]. Figure 1 (left) shows how a single faulty service can disrupt other interconnected services due to fault propagation. Sidecars monitor individual services and relay the information to the service mesh control plane to make dynamic decisions at runtime. Existing features of sidecars include traffic monitoring, load balancing, and fault-tolerant networking, but there is no support to *deeply* observe the “execution” of microservice at runtime to ensure consistency [4, 14].

Existing literature highlights the benefits of observing microservices at the kernel level using the extended Berkeley Packet Filter (eBPF) [4, 22, 13, 3, 18, 10]. eBPF offers a rich set of functionalities to monitor the runtime behaviour of microservices. It enables the service owners to develop *kernel probes*, which are user-defined programs with kernel privileges to deeply introspect the behaviour of applications running in the user space. eBPF provides several functionalities, including support for network observation, disk monitoring, and system call tracing to get fine-grained activity logs of user-level programs such as microservices. Extant literature proposes eBPF-based frameworks to classify microservices in data centres [3, 4] and for monitoring network and performance [22, 13, 18], lacking a solution to detect and suppress runtime faults in microservice-based applications, which is the focus of this work.

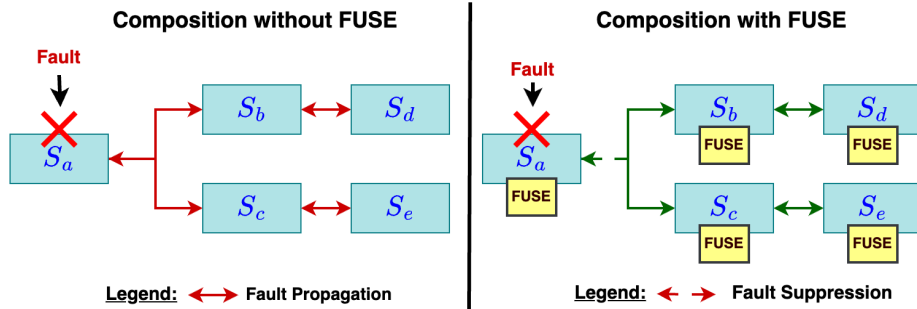


Fig. 1. Composing Microservices using FUSE to suppress fault.

This paper introduces *FUSE*, an eBPF-based fault-diagnosis and suppression framework for microservices architecture. FUSE differs from other works in the following ways:

- FUSE observes the runtime behaviour of microservices by monitoring system calls, function invocations, and disk accesses, including memory allocations, to create a *unique signature* (or digest) from traces for each invocation.
- FUSE categorises microservices as *strictly* or *partially* idempotent service based on the signature.
- FUSE detects runtime faults by checking the signature of each invocation against the expected signatures. Upon fault detection, it promptly alerts the service administrator and the circuit-breaker to suppress cascading failures, as shown in Figure 1 (right).
- FUSE also helps the service owners and developers quantify the runtime consistency of microservices through a novel *stability score* for microservices. A microservice composition with a low stability score is good for resiliency.

FUSE is implemented in a Linux environment with eBPF and evaluated AWS EC2 instances running the Ubuntu operating system. The performance and effectiveness of FUSE are validated using four representative microservices that 1) register a user into a MySQL database, 2) query the system to retrieve the list of users, 3) test the strength of a password, and 4) add the numbers in a list. During the evaluation, FUSE successfully detected 84 runtime faults, highlighting its ability to prevent cascading failures by alerting service administrators and circuit breakers. Lastly, the stability score is a valuable feature for service administrators and software developers to improve the microservices’ resiliency.

2 Background

2.1 extended Berkeley Packet Filter (eBPF)

The extended Berkeley Packet Filter, commonly known as eBPF, is a special type of virtual machine within the Linux kernel [15]. It was introduced in 1992 with a register-based filter for evaluating the network packets deep inside the kernel. *tcpdump* is one of the popular tools from the eBPF family. Although network monitoring is one of the popular use cases of eBPF, it does provide multiple hooks to trace and monitor various subsystems inside the Linux kernel. Some of the other hooks include Filetop (for tracking the file reads and writes), Opensnoop (attempts to track the files accessed by a specific process), and syscount (counts the number of system calls) [9]. The user-level programs can attach to these kernel hooks through eBPF probes such as kprobe, uprobe, tracepoint, and socket [9]. Depending on the type of selected hooks, the probes deliver detailed information to the user for deep introspection of application activities, including microservices.

2.2 Faults in Microservices

Microservices use a software framework such as Apache and Spring and run on a hardware infrastructure provided by the cloud provider in the case of cloud-native deployments. As discussed in [1], the microservices could be exposed to intermittent hardware faults due to cosmic radiation or impure packaging material. Facebook’s

data centres that serve a multitude of apps, such as Facebook, WhatsApp, and Instagram, experienced *silent data corruptions*, causing inaccurate computations [6]. Such runtime silent faults occur as a result of manufacturing defects at the silicon level [5]. The aging of microservices also introduces faults, which are undetected by Kubernetes probes, according to [17]. BROFY [10] explains the need to develop approaches for microservices’ integrity validation as the attackers or faulty hardware could introduce *bitflip errors* (one or more of the bits gets flipped, changing the runtime behaviour) to the computation infrastructure, causing silent and undetectable failures. These works discuss the importance of developing approaches to detect runtime faults in microservices, which is the focus of this work.

3 Related Work

Existing works have studied approaches to enhance the monitoring capabilities of microservices to detect performance issues [4, 22, 13, 3, 18]. These works leverage eBPF to observe microservices’ runtime behaviour by focusing on the networking activities [13], including TCP traffic [22] and latency [18], CPU activations [3, 22, 4], Block I/O performance [22] to understand the performance bottlenecks. Although these works aim to improve microservices’ performance, they don’t propose fault diagnosis or suppression.

MAGNet [3] is similar to FUSE in the aspect of application-focused eBPF tracing, but it aims to generate identities for workloads in data centres without detecting faults using the eBPF traces. Hyunseok *et al.* [4] introduced a microservices fingerprinting technique by tracing the system calls used by microservices and trained a machine learning model to classify services. FUSE classifies the microservices based on eBPF trace, but it uses system calls, disk I/O activities, and function invocations to generate a unique digest per invocation without employing machine learning. Furthermore, FUSE contributes a fault detection and suppression framework and a stability scoring mechanism to tackle runtime faults.

The stability of microservices is studied using eBPF in [20], which leverages variable autoencoders to detect unstable or compromised containers based on eBPF traces. It monitors a pre-configured set of 72 Linux system calls to capture specific security incidents and application faults. This work is similar to ours in the aspect of the eBPF-based approach for stability analysis, but we focus on application-level or microservices fingerprinting using system calls, function invocation, and disk accesses without involving any machine learning. Areeg and Claus [19] detect anomalies in containerised microservices using Markov Models, wherein the key performance indicators such as *mpstat* and *vmstat* are collected at the application level to learn a model using Hierarchical Hidden Markov Models, which detect abnormal microservice behaviours with 97% accuracy. Unlike this paper, FUSE generates microservice-specific eBPF traces and detects stability and abnormality through a hash-based signature without applying machine learning algorithms. Many other works tackle anomalies or eBPF-based observability for microservices, but they don’t focus on fault detection using eBPF-based tracing by combining system calls, function invocations, and disk accesses. Besides, none of the existing works introduces a stability scorecard for microservices, which is another novel contribution of our work.

4 FUSE: Fault Diagnosis and Suppression with eBPF for Microservices

4.1 System Model

Modern applications are composed of interconnected microservices, wherein each service, $s_x \in \mathcal{S}$, could be connected with one or more services. In such circumstances, whenever the end user issues a request to a service, it may trigger a series of services to generate a response to the user. Upon receiving a request, req_x , the service, s_x , processes the request by running computations, exe_x , and produces a response, res_x . Here, exe_x may involve contacting other services, meaning the response could be generated with the help of other dependent services. Each service in $\mathcal{S}$ gets invoked numerous times in deployment, depending on the application's popularity and the customer base. Each service gets invoked $\mathcal{K}$ ($\mathcal{K} \in \mathbb{N}$) times in deployment and the i^{th} invocation of a service, s_x , can be denoted by I_x^i . Each invocation includes the execution phase (exe_x^i), which will use the hardware and software resources, including the Linux kernel. eBPF provides tools to introspect the behaviour of a service, s_x , at the execution phase by capturing kernel-level traces for exe_x^i , which is denoted by $trace_x^i$. In summary, i^{th} invocation of a service, s_x , can be represented by a tuple $\langle req_x^i, exe_x^i, res_x^i, trace_x^i \rangle$. This work builds on some key concepts, such as idempotency and stability, which are defined below.

Definition 1. Idempotency *The system's state remains the same when an operation is executed once or any number of times [16].*

GET and *HEAD* are examples of idempotent HTTP operations because they don't change the server's state on the request's successful completion. Besides, even the same request can be issued many times without altering the server's state. On the other hand, the *POST* operation of HTTP changes the server's state, meaning it may add a new item to a database; hence, it is not idempotent.

Definition 2. Idempotent Microservice *A microservice is considered idempotent if it doesn't alter the system's state when a request is processed once or any number of times.*

A microservice that reads an employee's data from a database is an example of an idempotent microservice. In contrast, the service that registers a new employee's data is not idempotent as it changes the server's state.

Definition 3. Trace *A trace of a microservice corresponds to the low-level activities that the service carries out inside the kernel to process an API request. $\mathcal{T}_x$ denotes a set of **unique** traces of service, s_x , and $\mathcal{L}_x$ denotes the length of $\mathcal{T}_x$.*

Software applications access the CPU, memory, and network by invoking schedulers, I/O management modules, and network managers to fulfil the desired functionalities. Here, the low-level activities correspond to all activities that happen within the kernel as part of the application's or microservice's execution.

Definition 4. Strictly Idempotent Trace *A microservice's trace is strictly idempotent if it doesn't change when the same or different requests are processed once or any number of times.*

Considering three invocations, p , q , and r , of service, s_x , the trace is said to be idempotent if and only if:

$$trace_x^p = trace_x^q = trace_x^r = \alpha, \text{ where } \mathcal{T}_x = \{\alpha\} \text{ and } \mathcal{L}_x = 1 \quad (1)$$

For a strictly idempotent microservice, $\mathcal{T}_x$ includes only one unique trace, which is α in this case; thus, its length is always 1.

Definition 5. Partially Idempotent Trace A microservice's trace is said to be partially idempotent if it produces more than one trace consistently ($1 < \mathcal{L} < \tau$). In other words, a partially idempotent microservice produces multiple traces when handling requests but only produces up to τ number of unique traces.

The microservice s_x is partially idempotent when it consists of three known consistent traces α , β and γ in $\mathcal{T}_x$ (i.e., $\mathcal{T}_x = \{\alpha, \beta, \gamma\}$). Considering three invocations, p , q , and r , of service, s_x , the trace is said to be partially idempotent because $trace_x^p = \alpha, trace_x^q = \beta, trace_x^r = \gamma$. Here, $\mathcal{L}_x$ is 3, because $\mathcal{T}_x = \{\alpha, \beta, \gamma\}$. Repeated executions of s_x will always produce one of α , β or γ as traces, except when there is a fault.

Definition 6. Stability of a Microservice: A microservice is said to be stable if its traces are strictly or partially idempotent. The stability of a service in percentage is calculated using traces generated from $\mathcal{E}$ invocations. The stability of a microservice, sm_x , is:

$$sm_x = (1 - (1 \div \mathcal{L}_x)) * 100 \quad (2)$$

Figure 2 (top) shows an example of a strictly idempotent service with a stability score of 0% for $\mathcal{E}$ of 4 because it produces the same trace. On the other hand, when the service produces more than one trace for $\mathcal{E}$ ($\mathcal{E} \in \mathbb{N}$) invocations, then it is considered partially idempotent. Figure 2 (bottom) illustrates a partially idempotent service with a stability score of 50% for $\mathcal{E}$ of 4, indicating two unique traces α and β . The stability score of a strictly idempotent service is always lower than a partially idempotent service's stability score, assuming both services are invoked $\mathcal{E}$ times equally. Furthermore, the higher stability score indicates runtime inconsistency, helping developers and administrators to build highly stable services.

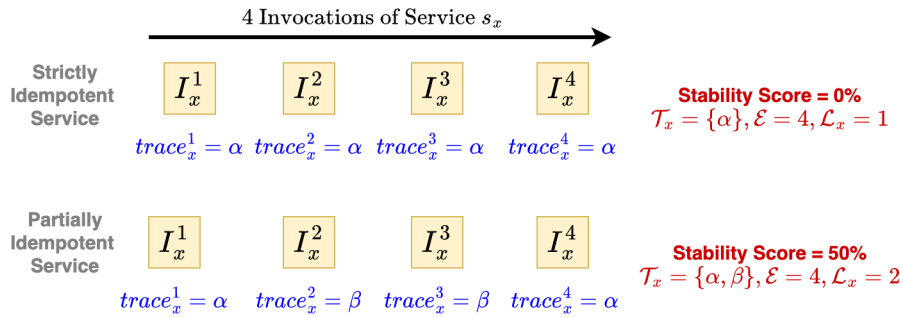


Fig. 2. Stability Score of Strictly and Partially Idempotent Service based on 5 Invocations.

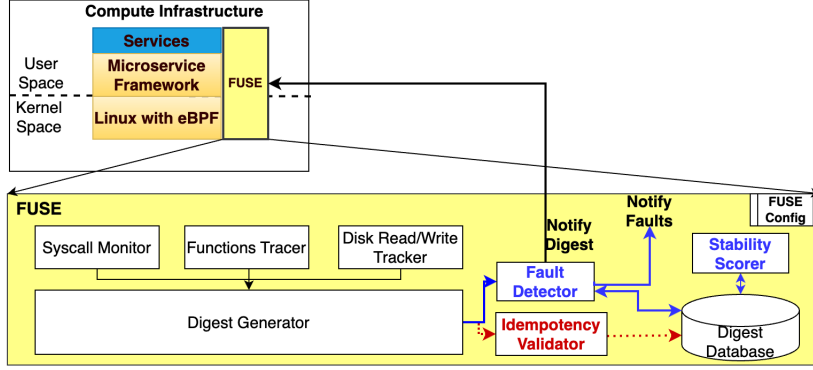


Fig. 3. FUSE Architecture: Normal line indicates operations in idempotency validation mode, while the dotted line denotes fault detection and suppression operations.

4.2 Architecture of FUSE

At a high level, FUSE is a cross-layer architecture running at user and kernel spaces, as shown in Figure 3. Microservices run in a computing environment, including a cloud instance. Typically, services are hosted on a framework such as Spring Boot. An operating system such as Ubuntu manages the resources for the application-level services. In this model, the operating system functions within the kernel space without offering much visibility to the users. In contrast, the services sit in the user space, leveraging the operating system to function correctly.

FUSE leverages eBPF to monitor and extract kernel space activities of services to generate signatures and suppress faults. The user space activities for eBPF include setting up the necessary hooks and the configurations to generate, process, store, and validate traces. In contrast, the kernel space intercepts the traces required for the target microservices. The fundamental building blocks of FUSE are discussed in the rest of the section.

FUSE Configurations FUSE can operate in two modes, including *idempotency validation* and *fault detection and suppression* modes, based on the configurations set by the microservices administrator. In the *idempotency validation mode*, FUSE runs the same microservice $\mathcal{E}$ times and collects the traces in a database. Subsequently, the traces are analysed to determine whether the microservice is *strictly* or *partially* idempotent. This mode must be activated for each microservice before production for fault diagnosis and suppression. The *fault detection and suppression mode* is activated at runtime to validate the microservices' trace for each invocation against the database. As shown in Figure 3, this mode uses a different set of building blocks for detecting and suppressing faults based on the runtime traces.

Syscall Monitor tracks the system calls used by the microservice. At each microservice's invocation, this building block intercepts the system calls and generates a list, including the counts for each system call. For example, read is one of the widespread system calls, and it opens a file or a resource. **Functions Tracer** traces

Algorithm 1: FUSE in idempotency validation and fault detection and suppression modes: Stability score calculation is included in idempotency validation mode.

FUSE in idempotency validation mode

Require: $S_x, \mathcal{E}$
 $\mathcal{T} \leftarrow \{\}$ */* Empty $\mathcal{T}$ */*

for $i = 1$ **to** $\mathcal{E}$ **do**
 $trace_i = \text{generateDigest}(S_x)$
 if $trace_i \notin \mathcal{T}$ **then**
 $\mathcal{T} = \mathcal{T} \cup trace_i$
 end if
end for
 $\text{storeTraceinDatabase}(S_x, \mathcal{T})$
return

FUSE Stability Score Calculation

Require: $S_x, \mathcal{T}$
 $\mathcal{L} \leftarrow 0$ */* Length of $\mathcal{T}$ is 0 */*
 $sm \leftarrow 0$ */* Stability score is initialised 0 */*
 $\mathcal{L} = |\mathcal{T}|$ */* Length of $\mathcal{T}$ */*
 $sm = (1 - (1 \div \mathcal{L})) * 100$ */* Stability score is calculated */*
 $\text{storeStabilityScoreinDatabase}(S_x, sm)$
return

FUSE in fault detection and suppression mode

$trace_j = \text{generateDigest}(S_x)$ */* j^{th} invocation of a service */*
if $trace_j \notin \mathcal{T}$ **then**
 $\text{notifyFault}()$
end if

the functions and libraries invoked by the microservice. For each invocation, this captures the list of files and libraries accessed by the microservice. **Disk Read/Write Tracker** tracks the number of reads and writes, including memory allocations, that happen during the execution of microservice along with a pointer to a file, indicating the files that were read from or written to.

Digest (or Signature) Generator processes the traces generated by the syscall monitor, functions tracer, and disk read/write tracker to create a digest or signature for each invocation. The digest is made by counting the number of calls, the list of functions invoked, the amount of data read from or written to, and *malloc* allocations. A cryptographic hash function such as sha256 creates a unique signature per invocation.

Idempotency Validator processes the digests during the *idempotency validation mode* by studying the consistency of digests. As mentioned in Section 4.2,

each microservice is executed $\mathcal{E}$ times to perform idempotency validation. Therefore, this process is data intensive as multiple traces must be generated to determine whether a given microservice is *strictly* or *partially* idempotent. A strictly idempotent microservice would always produce the same cryptographic hash because such a microservice always executes the same system calls, accesses the same list of functions, has the same data read from or written to, and allocates the same amount of memory. Algorithm 1 explains FUSE's operations.

Digest Database stores and manages digests associated with a microservice-based application. It is important to note that each microservice need not have its database; instead, a single database can be used to manage the digests.

Fault Detector gets activated in *fault detection and suppression mode*, and it is responsible for checking whether a digest of a recent microservice's invocation produces the known and expected digests (*strictly* vs. *partially* idempotent). Recall that a strictly idempotent microservice will have the same digest unless there is a fault. In contrast, a partially idempotent microservice will have one of the known finite traces if non-faulty (see Definition 7). This module checks the digest and generates an alert to the service admin. Besides, the digest is also notified to the circuit breaker for fault suppression.

4.3 Fault Diagnosis and Suppression

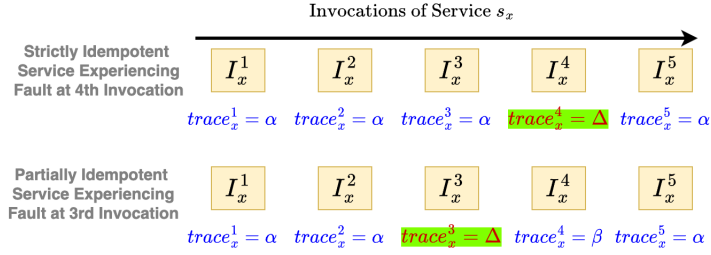


Fig. 4. Fault breaks the idempotency of both strictly and partially idempotent service.

Definition 7. Faulty Microservice A microservice is said to be faulty when its traces are *intermittently non-idempotent*. Both strictly idempotent and partially idempotent microservice can experience faults at runtime, breaking idempotency.

Figure 4 elucidates how a strictly idempotent microservice can experience an unexpected fault at runtime due to hardware or software failures, breaking the trace consistency. Note that any trace other than α is considered faulty for the strictly idempotent service in Figure 4 (top). On the other hand, a partially idempotent service must generate one of the known traces to be considered non-faulty; however, the 3rd invocation (see Figure 4 (bottom)) produces an unexpected trace of Δ , denoting a runtime fault.

Figure 1 (left) shows that a single faulty service could impact other interconnected services if left untreated. Circuit breakers are recommended to increase the resiliency and availability of microservices in the event of cascading failures. A circuit breaker relies on *fault event* to open the circuit to prevent cascading failures. Network-related issues, server failures, and overloads are considered fault events in circuit breakers, allowing microservices to overcome major and obvious faults. Our approach complements existing techniques and further strengthens the circuit breakers by tackling less-obvious faults, which could arise due to hardware abnormalities or any unexpected behaviour only noticeable at runtime, including the involvement of an adversary. Our fault suppression technique is presented in the next section.

4.4 Fault Suppression

FUSE digests (or signatures) provide a stable reference for fault diagnosis, as a service could be *strictly or partially idempotent*. Our fault suppression technique is proxy-based [2], meaning that the service interacts with other dependent services if and only if the digest of the current invocation satisfies the idempotency property (see Definitions 4 and 5). Any variations to the digest (or trace) indicate a fault (see Definition 7), suppressing outbound communications with dependent services, as shown in Figure 1 (right). The circuit breaker opens the connection to prevent cascading failures. The *fault detection and suppression* mode monitors the trace following Algorithm 1 and notifies faults to the appropriate agent. Following a proxy-based fault suppression scheme, the outgoing requests to dependent microservices are blocked to prevent cascading failures, as shown in Figure 1.

4.5 Stability Score

FUSE validates the stability of microservices through a stability scorer module, as shown in Figure 3. As discussed in Definition 6, the stability of a microservice depends on its trace (or signature) consistency. Our stability scoring mechanism provides a score between 0% and 100% for microservices. A score closer to 0% indicates high stability, while any score close to 100% indicates poor stability. Strictly idempotent services have a low stability score, while partially idempotent services have a high stability score ($> 0\%$). The stability scorer module takes the digests from the database and counts the unique digests per microservice in the *idempotency validation mode*, as shown in Algorithm 1.

This score is beneficial for system administrators and microservice architects. From the service management and resilience viewpoint, having a strictly idempotent microservice with a stability score of 0% maximises the determinism and fault detection capabilities. In contrast, any score above 0% shows the non-determinism of the microservice. Besides, when designing a microservice, the developers can aim to compose a *strictly idempotent microservice* by using FUSE’s stability score as it helps the developers understand hidden uncertainties in the code, which is only noticeable at runtime. Figure 2 shows how the stability score is calculated for strictly and partially idempotent microservices.

5 Proof-of-Concept Implementation and Evaluation

FUSE is implemented using eBPF and tested on AWS EC2 instances running the Ubuntu operating system. To validate the practicality of FUSE, we have developed POST and GET services using Python Flask with the following functionalities: **1) New user registration service (S1)**: receives a POST request with the user details, including the name, address, and country, and stores them in a *MySQL* database. **2) Users data retriever service (S2)**: handles a GET request to gather the users' data from a *MySQL* database and sends it back to the requester. **3) Password strength checker service (S3)**: receives a POST request to check the strength of a password the user selects. It receives a password string and checks its strength by assessing the lower and upper cases, digits, and length, and returns the password strength as *Strong* or *Weak*. **4) Addition service (S4)**: adds the numbers in the request and returns the sum as a response. Our proof-of-concept implementation used a proxy that generates and validates the trace (or signature) for each POST or GET request. We evaluate the performance and stability of the example representative services and report the results. Each service was executed more than 1000 times in *idempotency validation* mode, i.e. $\mathcal{E} > 1000$.

5.1 Idempotency of Example Services

For $\mathcal{E} > 1000$, services S1, S2, S3, and S4 have $\mathcal{T}$ of 4, 3, 3, and 9, respectively, making them partially idempotent. The sha256 hash is shortened to four characters in Table 1 for brevity. However, the real digest is 64 characters long. The amount of memory used, the number of functions accessed, and system calls invoked changes depend on the microservices, resulting in unique hashes per service. Figure 5 shows the distribution of hashes for S1, S2, S3, and S4 - it is clear that each microservice has a dominating hash that appears more than 97% of the time. Besides, S1, S2, S3, and S4 have stability scores of 50, 75, 66.6, and 90, respectively. The lower stability score corresponds to high determinism and stability. S1 is the most stable among the example services, while S4 is the least stable. Figure 6 shows how the stability score evolved with each invocation and stabilised. The instability of S4 and the high stability of S1 are apparent in Figure 6.

Service	$\mathcal{T}$	$\mathcal{L}$	Service Type	$\mathcal{E}$	Stability Score (in %)
S1	{'8507':7, 'f8ec':998}	2	Partially Idempotent	1005	50
S2	{'3173':2, '7eb0':1015, '913b':1, '51a7':4}	4	Partially Idempotent	1022	75
S3	{'075d':1094, '4532':11, '6947':1}	3	Partially Idempotent	1106	66.6
S4	{'05db':1, '23e2':3, '4d6b':6, '6398':1163, '68c0':1, '735d':4, '8801':2, 'e280':2, 'f541':7, 'd447':1}	10	Partially Idempotent	1190	90

Table 1. Stability Score of Services based on Traces.

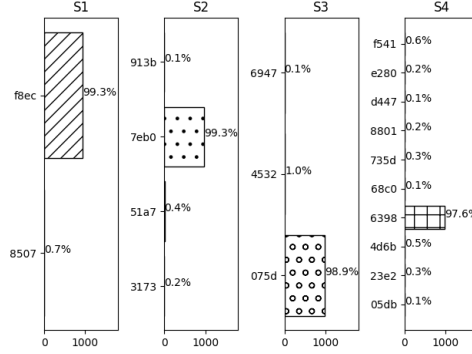


Fig. 5. Distribution of FUSE Traces for Services S1, S2, S3, and S4.

5.2 Overhead of FUSE

Traditional microservices don't rely on kernel-level traces for fault suppression. Thus, they don't incur any overhead within the request-response cycle. When a request is sent to a microservice, it gets processed, and the response is sent back. However, FUSE introduces an overhead in storage and latency. The **Storage Overhead of FUSE** originates from the storage of traces generated. FUSE produces multiple eBPF trace files to store file accesses, system call statistics, memory allocations, and disk operations. The storage overhead of FUSE per microservice execution is presented in Table 2, wherein the disk IO trace file takes up the most space while the system calls take up the least space, but these can be deleted periodically or immediately based on the requirements.

The **Latency of FUSE** differs from traditional microservices as the digest is generated immediately after the completion of each invocation. Without FUSE, S1, S2, S3, and S4 have an average latency of 13 milliseconds (ms), 14 ms, 5 ms, and 6 ms, respectively. In contrast, the average latency of S1, S2, S3, and S4 increases

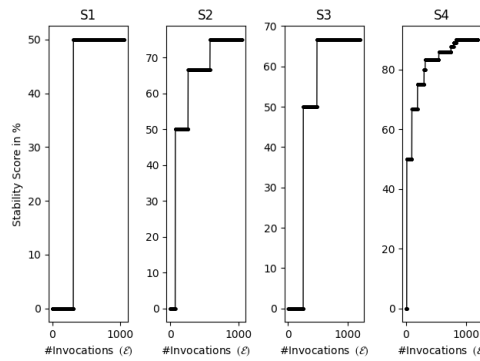


Fig. 6. Stability Score vs. Number of Invocations for S1, S2, S3, and S4.

Service	Syscall Trace (Bytes)	Function Trace (Bytes)	Disk I/O Trace (Bytes)
S1	751	1710	16380
S2	751	1710	16379
S3	751	1647	16380
S4	1039	1647	16385

Table 2. Storage Overhead for Trace Files

to 235 ms, 204 ms, 168 ms, and 276 ms with FUSE. The increased latency comes from processing the traces and the generation and notification of digest, which will be optimised in future work.

5.3 Faults Detected by FUSE

FUSE detected 84 faults at runtime for services S1 and S2 while S3 and S4 did not experience any faults, meaning all the traces for S3 and S4 came from $\mathcal{T}$ listed in Table 1. In contrast, for S1 and S2, FUSE detected faults with random traces that are unfound in $\mathcal{T}$ in Table 1. These 81 faults are because S1 and S2 rely on a MySQL database, which crashed due to being out of memory as *OOM killer* terminated the MySQL process, generating faulty traces. These traces indicated early signs of memory issues as they had additional system calls. The services ran correctly as long as there was enough memory, and then it started to experience faults, resulting in random and unknown traces, triggering faults. For S1, the observed traces during fault include (6805, 771a, 23fe), while for S2, the faulty traces include (dcdd, 1035). These faults underscore FUSE’s effectiveness in capturing runtime faults.

6 Discussion

Tool Selection for Digest Generation: eBPF provides a rich set of tools to generate traces of user-level programs, including microservices. This paper selects a few tools targeting system calls, function invocations, and disk operations. Table 1 shows that the microservices leave unique digests at the kernel level, indicating the effectiveness of the selected tools. However, there are more opportunities to expand the trace generation capabilities of FUSE to identify more hidden faults and runtime inconsistencies, which is an exciting direction for future work. **Impact of Inputs on the Digest:** Each invocation of service is unique due to the varying nature of inputs. Many microservices check the inputs’ validity before processing to prevent crashes. The evaluation presented in this paper considered different microservices with acceptable inputs. However, there are opportunities to carry out a deeper analysis of traces by varying the input drastically, which is also an interesting future work.

Platform-agnostic Traces: The evaluation leveraged AWS EC2 instances with an Ubuntu operating system to validate FUSE. There is an opportunity to run the same microservices on different types of eBPF-compatible Linux operating systems, such as Amazon Linux to understand whether the traces are platform-agnostic. This paper assumes that FUSE traces are generated in *idempotency validation* mode on

the platform where microservices will run in production. While this helps many real-world microservices, it would be good to strengthen FUSE by testing how the digests change when a new platform is introduced, as it would allow the administrator to migrate services to different platforms, considered future work. **Enhancing the Interface Efficiency of User and Kernel Spaces:** The proof-of-concept implementation leverages signals and timers to pass information between user and kernel space programs. While this demonstrates the effectiveness of FUSE, there is an opportunity to enhance the interface’s efficiency further. eBPF provides kprobes with customisation capabilities to improve the performance of FUSE, which requires developing novel extensions to eBPF. The authors plan to explore this in future work as it would reduce the overhead, including latency.

7 Conclusion

Microservices experience faults at runtime due to hardware issues, software bugs, and network interruptions. The ability to detect faults for microservice invocation would allow the administrators to better detect looming failures, including the prevention of cascading failures, as one faulty microservice could cause failures in multiple services. This paper introduced FUSE, a novel fault diagnosis and suppression tool based on eBPF. FUSE categorises microservices into *strictly* or *partially* idempotent services based on runtime traces generated by services. Furthermore, FUSE dynamically detects runtime faults by comparing the trace of each execution against the expected trace. Besides, it also prevents fault propagation by blocking external requests to other services whenever a fault is detected.

FUSE introduces a novel stability scoring mechanism for microservices by assigning a score to each microservice based on its trace consistency and the type of idempotency. The proof-of-concept implementation using eBPF and Flask and the deployment in AWS EC2 instances demonstrate FUSE’s practicality for real-world applications. FUSE’s performance evaluation results using four representative microservices show that many services have a set of unique digests; the services produce unknown and random hashes when there is a runtime fault, including out-of-memory issues. The successful detection of 84 runtime faults highlights the benefits of FUSE, but it introduces an overhead in the form of high latency and storage.

The future work includes the optimisation of FUSE’s performance by developing customised eBPF probes, validation of FUSE’s platform agnosticism by running the same services on different types of eBPF-compatible Linux platforms, and the inclusion of novel features to understand how inputs impact the digests by varying the inputs drastically for test services.

References

1. Cerveira, F., Oliveira, R.A., Barbosa, R., Madeira, H.: Evaluation of restful frameworks under soft errors. In: 2020 IEEE 31st International Symposium on Software Reliability Engineering (ISSRE). pp. 369–379. IEEE (2020)
2. Chandramouli, R.: Microservices-based application systems. NIST Special Publication 800(204), 800–204 (2019)

3. Chang, H., Kodialam, M., Lakshman, T.V., Mukherjee, S., Van der Merwe, J., Zaheer, Z.: Magnet: Machine learning guided application-aware networking for data centers. *IEEE Transactions on Cloud Computing* **11**(1), 291–307 (2023)
4. Chang, H., Kodialam, M., Lakshman, T., Mukherjee, S.: Microservice fingerprinting and classification using machine learning. In: 2019 IEEE 27th International Conference on Network Protocols (ICNP). pp. 1–11 (2019)
5. Constantinescu, C.: Intermittent faults and effects on reliability of integrated circuits. In: 2008 Annual Reliability and Maintainability Symposium. pp. 370–374. IEEE (2008)
6. Dixit, H.D., Pendharkar, S., Beadon, M., Mason, C., Chakravarthy, T., Muthiah, B., Sankar, S.: Silent data corruptions at scale. *arXiv preprint arXiv:2102.11245* (2021)
7. Dragoni, N., Giallorenzo, S., Lafuente, A.L., Mazzara, M., Montesi, F., Mustafin, R., Safina, L.: *Microservices: Yesterday, Today, and Tomorrow*, pp. 195–216. Springer International Publishing, Cham (2017)
8. Fulton III, S.M.: What led amazon to its own microservices architecture. *The New Stack* (2015)
9. Goldshtein, S.: *The next linux superpower: Ebpf primer*. USENIX Association: Dublin, Ireland (2016)
10. Hartono, A.P.P., Fetzer, C.: Brofy: Towards essential integrity protection for microservices. In: 2021 40th International Symposium on Reliable Distributed Systems (SRDS). pp. 154–163. IEEE (2021)
11. Jagadeesan, L.J., Mendiratta, V.B.: When failure is (not) an option: Reliability models for microservices architectures. In: 2020 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW). pp. 19–24. IEEE (2020)
12. Kakivaya, G., Xun, L., Hasha, R., Ahsan, S.B., Pfeiffer, T., Sinha, R., Gupta, A., Tarta, M., Fussell, M., Modi, V., et al.: Service fabric: a distributed platform for building microservices in the cloud. In: *Proceedings of the thirteenth EuroSys conference*. pp. 1–15 (2018)
13. Levin, J., Benson, T.A.: Viperprobe: Rethinking microservice observability with ebpf. In: 2020 IEEE 9th International Conference on Cloud Networking (CloudNet). pp. 1–8 (2020)
14. Li, W., Lemieux, Y., Gao, J., Zhao, Z., Han, Y.: Service mesh: Challenges, state of the art, and future research opportunities. In: 2019 IEEE International Conference on Service-Oriented System Engineering (SOSE). pp. 122–1225 (2019)
15. McCanne, S., Jacobson, V.: The bsd packet filter: A new architecture for user-level packet capture. In: *USENIX winter*. vol. 46 (1993)
16. *Microservices*, B.J., Varanasi, B., Bartkov, M.: *Spring REST*. Springer (2021)
17. Power, A., Kotonya, G.: A microservices architecture for reactive and proactive fault tolerance in iot systems. In: 2018 IEEE 19th International Symposium on "A World of Wireless, Mobile and Multimedia Networks" (WoWMoM). pp. 588–599 (2018)
18. Ranjitha, K., Tammana, P., Kannan, P.G., Naik, P.: A case for cross-domain observability to debug performance issues in microservices. In: 2022 IEEE 15th International Conference on Cloud Computing (CLOUD). pp. 244–246. IEEE (2022)
19. Samir, A., Pahl, C.: Dla: Detecting and localizing anomalies in containerized microservice architectures using markov models. In: 2019 7th International Conference on Future Internet of Things and Cloud (FiCloud). pp. 205–213 (2019)
20. Sharma, P., Porras, P., Cheung, S., Carpenter, J., Yegneswaran, V.: Scalable microservice forensics and stability assessment using variational autoencoders (2021)
21. Singleton, A.: The economics of microservices. *IEEE Cloud Computing* **3**(5), 16–20 (2016)
22. Weng, T., Yang, W., Yu, G., Chen, P., Cui, J., Zhang, C.: Kmon: An in-kernel transparent monitoring system for microservice systems with ebpf. In: 2021 IEEE/ACM International Workshop on Cloud Intelligence (CloudIntelligence). pp. 25–30 (2021)